

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу
«Операционные системы»

Группа: М8О-209БВ-24

Студент: Буланов Михаил

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 1.12.25

Москва, 2025

Постановка задачи

Вариант 3.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files).

Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

Системные вызовы для работы с отображаемыми файлами (file mapping):

- **open(path, oflag, mode)** — открывает или создаёт обычный файл, который используется как основа (backing file) для отображаемой памяти.
- **ftruncate(fd, length)** — устанавливает размер файла, обеспечивая достаточное место для размещения структуры разделяемой памяти.
- **mmap(addr, length, prot, flags, fd, offset)** — отображает файл в адресное пространство процесса, позволяя нескольким процессам совместно использовать один и тот же участок памяти.
- **munmap(addr, length)** — снимает отображение памяти, освобождая ресурсы.
- **close(fd)** — закрывает файловый дескриптор после того, как файл был отображён.

Системные вызовы для работы с процессами:

- **fork()** — создаёт дочерний процесс, копируя контекст родителя.
- **execl(path, arg0, arg1, ..., NULL)** — заменяет текущий образ процесса новым, загружая исполняемую программу (child1, child2).
- **waitpid(pid, status, options)** — ожидает завершения дочернего процесса и получает его код выхода.
- **exit(status)** — завершает выполнение процесса с указанным кодом.

Системные вызовы для обработки сигналов:

- **sigaction(signo, act, oldact)** — устанавливает новый обработчик сигнала (SIGUSR1, SIGUSR2) для синхронизации процессов.
- **kill(pid, signo)** — отправляет указанный сигнал другому процессу.
- **pause()** — приостанавливает процесс до получения сигнала.

Алгоритм работы программы:

1. Инициализация родительского процесса

- Создаётся файл, который будет использоваться как область совместной памяти всеми процессами.
- Через вызов `ftruncate()` файл расширяется до размера структуры `shared memory`.
- Файл отображается в адресное пространство родительского процесса через `mmap()`, что позволяет работать с памятью как с обычным массивом.
- Буфер и управляющие поля структуры инициализируются нулями (`stage = 0`).
- Устанавливается обработчик сигнала `SIGUSR2`, который будет использоваться для уведомления родителя о завершении обработки строки дочерними процессами.

2. Создание дочерних процессов

- Через `fork()` создаётся первый дочерний процесс (`Child1`).
- В дочернем процессе образ заменяется программой `child1` с помощью `execl()`.
- Аналогично создаётся второй дочерний процесс (`Child2`).
- После запуска каждый дочерний процесс выполняет собственное подключение к общей памяти, вызывая `mmap()` к тому же файлу.
- `Child1` и `Child2` устанавливают обработчики сигнала `SIGUSR1`, который используется для уведомления о готовности данных.

3. Взаимодействие процессов через разделяемую память и сигналы

Процессы взаимодействуют последовательно в строгой цепочке:

Parent → Child → Parent → Child → out.txt.

Работа родительского процесса (инициализация)

- Настраивает обработчик сигнала `SIGUSR1` (используется как уведомление).
- Создает два сегмента общей памяти на базе файлов и отображает их через `mmap()`:
 - `cmd.bin` (структура `cmd_data_t`) для передачи команд ребёнку,
 - `resp.bin` (структура `resp_data_t`) для получения ответа от ребёнка.
- Инициализирует два межпроцессных семафора (`pshared=1`):
 - `sem_cmd` (родитель → ребёнок: “команда готова”),
 - `sem_resp` (ребёнок → родитель: “ответ готов”).
- Запрашивает у пользователя имя файла результатов и записывает его в `cmd->command`.
- Создает дочерний процесс (`fork`), затем в дочернем выполняет `exec("./child", ...)`.
- Сохраняет PID ребёнка в `cmd->child_pid`.
- Ожидает “READY” от ребёнка (`sem_wait` на `sem_resp`) и выводит ответ.

Работа дочернего процесса (старт)

- Настраивает обработчик `SIGUSR1`.
- Открывает существующие `cmd.bin` и `resp.bin` и отображает их через `mmap()`.
- Открывает файл результатов на запись (имя берёт из `cmd->command`, которое родитель записал до `fork/exec`).

- Записывает в файл свой PID.
- Формирует ответ "READY":
 - resp->response = "READY",
 - resp->critical = 0,
 - sem_post(sem_resp),
 - отправляет родителю SIGUSR1 (как уведомление).

Работа родителя (основной цикл)

- Считывает от пользователя строку произвольной длины (команду).
- Если введён EOF (Ctrl + D), автоматически подставляет команду "exit".
- Пустые строки пропускает.
- Записывает введённую строку в cmd->command.
- Сигнализирует ребёнку о готовности команды:
 - sem_post(sem_cmd),
 - дополнительно посылает SIGUSR1 ребёнку.
- Ожидает ответ ребёнка: sem_wait(sem_resp).
- Выводит resp->response пользователю.
- Если resp->critical == 1 (критическая ошибка, деление на ноль):
 - выводит сообщение об ошибке,
 - завершает ребёнка (SIGTERM),
 - выполняет waitpid() и завершает работу.
- Если команда была "exit":
 - выполняет waitpid() и завершает цикл.

Работа дочернего процесса (обработка команды)

- Ожидает команду от родителя: sem_wait(sem_cmd).
- Если cmd->command == "exit":
 - пишет в файл "Получена команда выхода",
 - resp->response = "EXIT", resp->critical = 0,
 - sem_post(sem_resp) и SIGUSR1 родителю,
 - завершает работу.
- Иначе:
 - пишет в файл полученную команду,
 - извлекает из строки до 100 целых чисел (нечисловые токены игнорируются),
 - если чисел меньше 2: resp->response = "ERROR: нужно 2+ числа", resp->critical = 0,
 - если чисел 2 и более: выполняет последовательное деление (первое число делится на каждое следующее):
 - при делении на 0: resp->response = "CRITICAL: деление на ноль", resp->critical = 1, прекращает обработку,
 - иначе пишет шаги вычислений в файл и по завершении выдаёт resp->response = "OK", resp->critical = 0.
- После формирования ответа:

- sem_post(sem_resp),
- отправляет родителю SIGUSR1,
- при resp->critical == 1 завершает работу (выходит из цикла).

Завершение работы и освобождение ресурсов

- Нормальное завершение:
 - родитель отправляет команду "exit" (в том числе при Ctrl + D),
 - ребёнок возвращает "EXIT" и корректно завершается,
 - родитель вызывает waitpid() для ожидания ребёнка.
- Аварийное завершение при делении на ноль:
 - ребёнок ставит resp->critical = 1,
 - родитель завершает ребёнка SIGTERM и делает waitpid().
- Освобождение ресурсов:
 - родитель уничтожает семафоры (sem_destroy),
 - снимает отображение памяти (munmap),
 - удаляет файлы cmd.bin и resp.bin (unlink),
 - ребёнок закрывает файл результатов (fclose) и снимает mmap (munmap).

Код программы

```
common.h
#ifndef COMMON_H
#define COMMON_H
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <signal.h>
#include <string.h>
#include <sys/wait.h>
#include <ctype.h>
#include <errno.h>
#include <semaphore.h>
#define CMD_FILE "cmd.bin"
#define RESP_FILE "resp.bin"
#define MAXLINE 1024
typedef struct {
    sem_t sem_cmd;      // parent -> child
    sem_t sem_resp;     // child -> parent
    pid_t child_pid;
    char command[MAXLINE];
} cmd_data_t;
typedef struct {
    int critical;       // 1 if critical error
    char response[MAXLINE];
} resp_data_t;
static volatile sig_atomic_t sigusr1_seen = 0;
static void sigusr1_handler(int signum) {
    (void)signum;
    sigusr1_seen = 1;
}
```

```

static void* map_file(const char* name, size_t size, int create) {
    int fd = open(name, O_RDWR | (create ? (O_CREAT | O_TRUNC) : 0), 0600);
    if (fd < 0) return NULL;
    if (create) {
        if (ftruncate(fd, (off_t)size) == -1) {
            close(fd);
            return NULL;
        }
    }
    void* p = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    close(fd);
    return (p == MAP_FAILED) ? NULL : p;
}
#endif
parent.c
#include "common.h"
static void cleanup(cmd_data_t* cmd, resp_data_t* resp) {
    if (cmd) {
        sem_destroy(&cmd->sem_cmd);
        sem_destroy(&cmd->sem_resp);
        munmap(cmd, sizeof(*cmd));
    }
    if (resp) {
        munmap(resp, sizeof(*resp));
    }
    unlink(CMD_FILE);
    unlink(ESP_FILE);
}
int main(void) {
    signal(SIGUSR1, sigusr1_handler);
    cmd_data_t* cmd = (cmd_data_t*)map_file(CMD_FILE, sizeof(cmd_data_t), 1);
    resp_data_t* resp = (resp_data_t*)map_file(ESP_FILE, sizeof(resp_data_t), 1);
    if (!cmd || !resp) {
        perror("mmap");
        cleanup(cmd, resp);
        return 1;
    }
    memset(cmd, 0, sizeof(*cmd));
    memset(resp, 0, sizeof(*resp));
    if (sem_init(&cmd->sem_cmd, 1, 0) == -1) {
        perror("sem_init sem_cmd");
        cleanup(cmd, resp);
        return 1;
    }
    if (sem_init(&cmd->sem_resp, 1, 0) == -1) {
        perror("sem_init sem_resp");
        cleanup(cmd, resp);
        return 1;
    }
    printf("Введите имя файла для результатов: ");
    fflush(stdout);
    if (!fgets(cmd->command, MAXLINE, stdin)) {
        fprintf(stderr, "Ошибка чтения имени файла.\n");
        cleanup(cmd, resp);
        return 1;
    }
    cmd->command[strcspn(cmd->command, "\n")] = '\0';
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        cleanup(cmd, resp);
        return 1;
    }

```

```

}
if (pid == 0) {
    execl("./child", "./child", NULL);
    perror("execl");
    _exit(1);
}
cmd->child_pid = pid;
// wait READY
if (sem_wait(&cmd->sem_resp) == -1) {
    perror("sem_wait READY");
    cleanup(cmd, resp);
    return 1;
}
printf("Child says: %s\n", resp->response);
char line[MAXLINE];
while (1) {
    printf("Введите команду (или Ctrl+D для exit): ");
    fflush(stdout);
    if (!fgets(line, sizeof(line), stdin)) {
        // EOF -> exit
        strncpy(cmd->command, "exit", MAXLINE - 1);
        cmd->command[MAXLINE - 1] = '\0';
    } else {
        line[strcspn(line, "\n")] = '\0';
        if (line[0] == '\0') continue; // skip empty
        strncpy(cmd->command, line, MAXLINE - 1);
        cmd->command[MAXLINE - 1] = '\0';
    }
    if (sem_post(&cmd->sem_cmd) == -1) {
        perror("sem_post sem_cmd");
        break;
    }
    kill(cmd->child_pid, SIGUSR1);
    if (sem_wait(&cmd->sem_resp) == -1) {
        perror("sem_wait sem_resp");
        break;
    }
    printf("Ответ: %s\n", resp->response);
    if (resp->critical) {
        fprintf(stderr, "Критическая ошибка. Завершаем child.\n");
        kill(cmd->child_pid, SIGTERM);
        waitpid(cmd->child_pid, NULL, 0);
        cleanup(cmd, resp);
        return 1;
    }
    if (strcmp(cmd->command, "exit") == 0) {
        waitpid(cmd->child_pid, NULL, 0);
        break;
    }
}
cleanup(cmd, resp);
return 0;
}

```

child.c

```

#include "common.h"
int main(void) {
    signal(SIGUSR1, sigusr1_handler);
    cmd_data_t* cmd = (cmd_data_t*)map_file(CMD_FILE, sizeof(cmd_data_t), 0);
    resp_data_t* resp = (resp_data_t*)map_file(RESF_FILE, sizeof(resp_data_t), 0);
    if (!cmd || !resp) {
        perror("child mmap");
    }
}

```

```

    return 1;
}
// cmd->command содержит имя файла результатов (записано родителем до fork/exec)
FILE* out = fopen(cmd->command, "w");
if (!out) {
    snprintf(resp-
>response, MAXLINE, "ERROR: cannot open output file: %s", strerror(errno));
    resp->critical = 0;
    sem_post(&cmd->sem_resp);
    kill(getppid(), SIGUSR1);
    munmap(cmd, sizeof(*cmd));
    munmap(resp, sizeof(*resp));
    return 1;
}
fprintf(out, "Child PID: %d\n", getpid());
fflush(out);
strncpy(resp->response, "READY", MAXLINE - 1);
resp->response[MAXLINE - 1] = '\0';
resp->critical = 0;
sem_post(&cmd->sem_resp);
kill(getppid(), SIGUSR1);
while (1) {
    if (sem_wait(&cmd->sem_cmd) == -1) {
        perror("child sem_wait sem_cmd");
        break;
    }
    char local[MAXLINE];
    strncpy(local, cmd->command, MAXLINE - 1);
    local[MAXLINE - 1] = '\0';
    if (strcmp(local, "exit") == 0) {
        fprintf(out, "Получена команда выхода\n");
        fflush(out);
        strncpy(resp->response, "EXIT", MAXLINE - 1);
        resp->response[MAXLINE - 1] = '\0';
        resp->critical = 0;
        sem_post(&cmd->sem_resp);
        kill(getppid(), SIGUSR1);
        break;
    }
    fprintf(out, "Команда: %s\n", local);
    // parse integers
    int nums[100];
    int n = 0;
    char tmp[MAXLINE];
    strncpy(tmp, local, MAXLINE - 1);
    tmp[MAXLINE - 1] = '\0';
    for (char* tok = strtok(tmp, " \t"); tok && n < 100; tok = strtok(NULL, " \t")) {
        char* end = NULL;
        long v = strtol(tok, &end, 10);
        if (end != tok && *end == '\0') {
            nums[n++] = (int)v;
        }
    }
    if (n < 2) {
        strncpy(resp->response, "ERROR: нужно 2+ числа", MAXLINE - 1);
        resp->response[MAXLINE - 1] = '\0';
        resp->critical = 0;
        fprintf(out, "Ошибка: нужно 2+ числа\n");
        fflush(out);
        sem_post(&cmd->sem_resp);
        kill(getppid(), SIGUSR1);
        continue;
    }
}

```



```

    }
    long result = nums[0];
    int critical = 0;
    for (int i = 1; i < n; i++) {
        if (nums[i] == 0) {
            critical = 1;
            fprintf(out, "CRITICAL: деление на ноль\n");
            break;
        }
        long prev = result;
        result /= nums[i];
        fprintf(out, "%ld / %d = %ld\n", prev, nums[i], result);
    }
    fflush(out);
    if (critical) {
        strncpy(resp->response, "CRITICAL: деление на ноль", MAXLINE - 1);
        resp->response[MAXLINE - 1] = '\0';
        resp->critical = 1;
        sem_post(&cmd->sem_resp);
        kill(getppid(), SIGUSR1);
        break;
    } else {
        strncpy(resp->response, "OK", MAXLINE - 1);
        resp->response[MAXLINE - 1] = '\0';
        resp->critical = 0;
        sem_post(&cmd->sem_resp);
        kill(getppid(), SIGUSR1);
    }
}
fclose(out);
munmap(cmd, sizeof(*cmd));
munmap(resp, sizeof(*resp));
return 0;
}

```

Протокол работы программы

Тестирование:

Файл для результатов: ans

Child says: READY

Дочерний PID: 645

Вводите числа через пробел ('exit' для выхода):

> 16 4 4 1

Ответ: OK

> 7 3 1 0

Ответ: CRITICAL: деление на ноль

```
execve("./parent", ["/.parent"], 0x7ffd4e344c68 /* 28 vars */) = 0
brk(NULL) = 0x55c4a863f000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffeb2a9d6e0) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f822b344000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=12460, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 12460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f822b340000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"..., 48, 848) =
48
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\00{\f\225\\=\201\327\312\301P\32$\230\266\235"...,
68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f822b117000
mprotect(0x7f822b13f000, 2023424, PROT_NONE) = 0
mmap(0x7f822b13f000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x28000) = 0x7f822b13f000
```

```

mmap(0x7f822b2d4000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000)
= 0x7f822b2d4000
mmap(0x7f822b32d000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x215000) = 0x7f822b32d000
mmap(0x7f822b333000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0x7f822b333000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f822b114000
arch_prctl(ARCH_SET_FS, 0x7f822b114740) = 0
set_tid_address(0x7f822b114a10) = 1747
set_robust_list(0x7f822b114a20, 24) = 0
rseq(0x7f822b1150e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f822b32d000, 16384, PROT_READ) = 0
mprotect(0x55c4a7452000, 4096, PROT_READ) = 0
mprotect(0x7f822b37e000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f822b340000, 12460) = 0
[SIGNALS] rt_sigaction(SIGUSR1, {sa_handler=0x55c4a7450509, sa_mask=[], sa_flags=SA_RESTOR
ER|SA_RESTART, sa_restorer=0x7f822b159520}, NULL, 8) = 0
[SHARED FILES] openat(AT_FDCWD, "cmd.bin", O_RDWR|O_CREAT|O_TRUNC, 0600) = 3
[SHARED FILES] ftruncate(3, 1096) = 0
[SHARED FILES] mmap(NULL, 1096, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f822b37d000
[SHARED FILES] close(3) = 0
[SHARED FILES] openat(AT_FDCWD, "resp.bin", O_RDWR|O_CREAT|O_TRUNC, 0600) = 3
[SHARED FILES] ftruncate(3, 1028) = 0
[SHARED FILES] mmap(NULL, 1028, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f822b343000
[SHARED FILES] close(3) = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH) =
0
getrandom("\xf7\xfa\xbb\x45\xd1\x79\xa4\x1c", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x55c4a863f000
brk(0x55c4a8660000) = 0x55c4a8660000
write(1, "\320\244\320\260\320\271\320\273 \320\264\320\273\321\217 \321\200\320\265\320\2
67\321\203\320\273\321\214\321\202\320\260"..., 40Файл для результатов: ) = 40
newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH) =
0
read(0, ans
"ans \n", 1024) = 5
[CHILD PROC] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD
strace: Process 1785 attached
, child_tidptr=0x7f822b114a10) = 1785
[pid 1785] set_robust_list(0x7f822b114a20, 24 <unfinished ...>
[SEM/FUTEX] [pid 1747] futex(0x7f822b37d428, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, N
ULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
[pid 1785] <... set_robust_list resumed>) = 0
[CHILD PROC] [pid 1785] execve("./child", ["child"], 0x7ffeb2a9d8b8 /* 28 vars */) = 0
[pid 1785] brk(NULL) = 0x564aed05f000
[pid 1785] arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe7c530850) = -
1 EINVAL (Invalid argument)
[pid 1785] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -
1, 0) = 0x7fcfd2933000
[pid 1785] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
[pid 1785] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 1785] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=12460, ...}, AT_EMPTY_PATH) =
0
[pid 1785] mmap(NULL, 12460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fcfd292f000
[pid 1785] close(3) = 0
[pid 1785] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 1785] read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\>\0\1\0\0\0P\237\2\0\0\0\0"...,
832) = 832
[pid 1785] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784

```

```

[pid 1785] pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0"...,
, 48, 848) = 48
[pid 1785] pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\00{\f\225\\=\201\327\312\301P\32$\230
\266\235"..., 68, 896) = 68
[pid 1785] newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH)
= 0
[pid 1785] pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
[pid 1785] mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fcfd27060
00
[pid 1785] mprotect(0x7fcfd272e000, 2023424, PROT_NONE) = 0
[pid 1785] mmap(0x7fcfd272e000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_D
ENYWRITE, 3, 0x28000) = 0x7fcfd272e000
[pid 1785] mmap(0x7fcfd28c3000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3
, 0x1bd000) = 0x7fcfd28c3000
[pid 1785] mmap(0x7fcfd291c000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DE
NYWRITE, 3, 0x215000) = 0x7fcfd291c000
[pid 1785] mmap(0x7fcfd2922000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_AN
ONYMOUS, -1, 0) = 0x7fcfd2922000
[pid 1785] close(3) = 0
[pid 1785] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -
1, 0) = 0x7fcfd2703000
[pid 1785] arch_prctl(ARCH_SET_FS, 0x7fcfd2703740) = 0
[pid 1785] set_tid_address(0x7fcfd2703a10) = 1785
[pid 1785] set_robust_list(0x7fcfd2703a20, 24) = 0
[pid 1785] rseq(0x7fcfd27040e0, 0x20, 0, 0x53053053) = 0
[pid 1785] mprotect(0x7fcfd291c000, 16384, PROT_READ) = 0
[pid 1785] mprotect(0x564aeba9d000, 4096, PROT_READ) = 0
[pid 1785] mprotect(0x7fcfd296d000, 8192, PROT_READ) = 0
[pid 1785] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY
}) = 0
[pid 1785] munmap(0x7fcfd292f000, 12460) = 0
[SIGNALS] [pid 1785] rt_sigaction(SIGUSR1, {sa_handler=0x564aeba9b489, sa_mask=[], sa_fla
gs=SA_RESTORER|SA_RESTART, sa_restorer=0x7fcfd2748520}, NULL, 8) = 0
[SHARED FILES] [pid 1785] openat(AT_FDCWD, "cmd.bin", O_RDWR) = 3
[SHARED FILES] [pid 1785] mmap(NULL, 1096, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f
cfd296c000
[SHARED FILES] [pid 1785] close(3) = 0
[SHARED FILES] [pid 1785] openat(AT_FDCWD, "resp.bin", O_RDWR) = 3
[SHARED FILES] [pid 1785] mmap(NULL, 1028, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f
cfd2932000
[SHARED FILES] [pid 1785] close(3) = 0
[pid 1785] getrandom("\x06\xcf\xfa\x5e\x9e\x02\x96\x57", 8, GRND_NONBLOCK) = 8
[pid 1785] brk(NULL) = 0x564aed05f000
[pid 1785] brk(0x564aed080000) = 0x564aed080000
[OUTPUT FILE] [pid 1785] openat(AT_FDCWD, "ans ", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
[pid 1785] getpid() = 1785
[pid 1785] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=0, ...}, AT_EMPTY_PATH) = 0
[OUTPUT FILE] [pid 1785] write(3, "\320\224\320\276\321\207\320\265\321\200\320\275\320\2
70\320\271 \320\277\321\200\320\276\321\206\320\265\321\201\321\201 "..., 43) = 43
[SEM/FUTEX] [pid 1785] futex(0x7fcfd296c428, FUTEX_WAKE, 1 <unfinished ...>
[SEM/FUTEX] [pid 1747] <... futex resumed> = 0
[SEM/FUTEX] [pid 1785] <... futex resumed> = 1
[pid 1747] write(1, "Child says: READY\n", 18 <unfinished ...>
Child says: READY
[pid 1785] getppid( <unfinished ...>
[pid 1747] <... write resumed> = 18
[pid 1785] <... getppid resumed> = 1747
[pid 1747] write(1, "\320\224\320\276\321\207\320\265\321\200\320\275\320\270\320\271 PID
: 1785\n", 27 <unfinished ...>
Дочерний PID: 1785
[SIGNALS] [pid 1785] kill(1747, SIGUSR1 <unfinished ...>

```

```

[pid 1747] <... write resumed>          = 27
[ SIGNALS ] [pid 1785] <... kill resumed>      = 0
[ SIGNALS ] [pid 1747] --
- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=1785, si_uid=0} ---
[pid 1785] futex(0x7fcfd296c408, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_B
ITSET_MATCH_ANY <unfinished ...>
[ SIGNALS ] [pid 1747] rt_sigreturn({mask=[]})      = 27
[pid 1747] write(1, "\320\222\320\262\320\276\320\264\320\270\321\202\320\265 \321\207\32
0\270\321\201\320\273\320\260 \321\207\320\265\321\200"... , 80Вводите числа через пробел (
'exit' для выхода):
) = 80
[pid 1747] write(1, "> ", 2) = 2
[pid 1747] read(0, 16 2 2
"16 2 2\n", 1024) = 7
[ SEM/FUTEX ] [pid 1747] futex(0x7f822b37d408, FUTEX_WAKE, 1) = 1
[ SEM/FUTEX ] [pid 1785] <... futex resumed>      = 0
[ SIGNALS ] [pid 1747] kill(1785, SIGUSR1 <unfinished ...>
[ OUTPUT FILE ] [pid 1785] write(3, "\320\232\320\276\320\274\320\260\320\275\320\264\320\2
60: 16 2 2\n\320\222\321\213\321\207\320\270\321"... , 93 <unfinished ...>
[ SIGNALS ] [pid 1747] <... kill resumed>          = 0
[ SEM/FUTEX ] [pid 1747] futex(0x7f822b37d428, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, N
ULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
[ OUTPUT FILE ] [pid 1785] <... write resumed>      = 93
[ SIGNALS ] [pid 1785] --
- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=1747, si_uid=0} ---
[ SIGNALS ] [pid 1785] rt_sigreturn({mask=[]})      = 93
[ SEM/FUTEX ] [pid 1785] futex(0x7fcfd296c428, FUTEX_WAKE, 1 <unfinished ...>
[ SEM/FUTEX ] [pid 1747] <... futex resumed>      = 0
[ SEM/FUTEX ] [pid 1785] <... futex resumed>      = 1
[pid 1785] getppid( <unfinished ...>
[pid 1747] write(1, "\320\236\321\202\320\262\320\265\321\202: OK\n", 15 <unfinished ...>
[pid 1785] <... getppid resumed>          = 1747
Ответ: OK
[pid 1747] <... write resumed>          = 15
[pid 1747] write(1, "> ", 2 <unfinished ...>
> [pid 1785] kill(1747, SIGUSR1 <unfinished ...>
[pid 1747] <... write resumed>          = 2
[pid 1785] <... kill resumed>          = 0
[ SIGNALS ] [pid 1747] --
- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=1785, si_uid=0} ---
[pid 1785] futex(0x7fcfd296c408, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_B
ITSET_MATCH_ANY <unfinished ...>
[ SIGNALS ] [pid 1747] rt_sigreturn({mask=[]})      = 2
[pid 1747] read(0, 2 2 0
"2 2 0\n", 1024) = 6
[ SEM/FUTEX ] [pid 1747] futex(0x7f822b37d408, FUTEX_WAKE, 1) = 1
[ SEM/FUTEX ] [pid 1785] <... futex resumed>      = 0
[ SIGNALS ] [pid 1747] kill(1785, SIGUSR1 <unfinished ...>
[ OUTPUT FILE ] [pid 1785] write(3, "\320\232\320\276\320\274\320\260\320\275\320\264\320\2
60: 2 2 0\n\320\222\321\213\321\207\320\270\321\201"... , 94 <unfinished ...>
[ SIGNALS ] [pid 1747] <... kill resumed>          = 0
[ SEM/FUTEX ] [pid 1747] futex(0x7f822b37d428, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, N
ULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
[ OUTPUT FILE ] [pid 1785] <... write resumed>      = 94
[ SIGNALS ] [pid 1785] --
- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=1747, si_uid=0} ---
[ SIGNALS ] [pid 1785] rt_sigreturn({mask=[]})      = 94
[ SEM/FUTEX ] [pid 1785] futex(0x7fcfd296c428, FUTEX_WAKE, 1) = 1
[ SEM/FUTEX ] [pid 1747] <... futex resumed>      = 0
[pid 1747] write(1, "\320\236\321\202\320\262\320\265\321\202: CRITICAL: \320\264\320\265
\320\273\320\265\320\275"... , 51 <unfinished ...>
Ответ: CRITICAL: деление на ноль

```

```

[pid 1785] getppid( <unfinished ...>
[pid 1747] <... write resumed>          = 51
[pid 1785] <... getppid resumed>        = 1747
[pid 1747] write(1, "\n", 1
) = 1
[SIGNALS] [pid 1785] kill(1747, SIGUSR1 <unfinished ...>
[pid 1747] write(1, "!!! \320\236\320\250\320\230\320\221\320\232\320\220: \320\224\320\2
25\320\233\320\225\320\235\320\230\320\225"... , 51 <unfinished ...>
[SIGNALS] [pid 1785] <... kill resumed>          = 0
[pid 1747] <... write resumed>          = ? ERESTARTSYS (To be restarted if SA_RESTART is
set)
[OUTPUT FILE] [pid 1785] close(3 <unfinished ...>
[SIGNALS] [pid 1747] --
- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=1785, si_uid=0} ---
[SIGNALS] [pid 1747] rt_sigreturn({mask=[]})      = 1
[pid 1747] write(1, "!!! \320\236\320\250\320\230\320\221\320\232\320\220: \320\224\320\2
25\320\233\320\225\320\235\320\230\320\225"... , 51!!! ОШИБКА: ДЕЛЕНИЕ НА НОЛЬ !!!
) = 51
[OUTPUT FILE] [pid 1785] <... close resumed>      = 0
[pid 1747] write(1, "\320\227\320\260\320\262\320\265\321\200\321\210\320\265\320\275\320
\270\320\265 \321\200\320\260\320\261\320\276\321\202\321"... , 37 <unfinished ...>
Завершение работы...
[SHARED FILES] [pid 1785] munmap(0x7fcfd296c000, 1096 <unfinished ...>
[pid 1747] <... write resumed>          = 37
[SIGNALS] [pid 1747] kill(1785, SIGTERM)         = 0
[SIGNALS] [pid 1747] wait4(1785, <unfinished ...>
[SHARED FILES] [pid 1785] <... munmap resumed>     = 0
[SIGNALS] [pid 1785] --
- SIGTERM {si_signo=SIGTERM, si_code=SI_USER, si_pid=1747, si_uid=0} ---
[SIGNALS] [pid 1785] +++ killed by SIGTERM +++
[CHILD PROC] <... wait4 resumed>NULL, 0, NULL)    = 1785
[SIGNALS] --
- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_KILLED, si_pid=1785, si_uid=0, si_status=SIGTERM,
si_ftime=0, si_stime=0} ---
[SHARED FILES] munmap(0x7f822b37d000, 1096)      = 0
[SHARED FILES] munmap(0x7f822b343000, 1028)      = 0
[SHARED FILES] unlink("cmd.bin")                = 0
[SHARED FILES] unlink("resp.bin")               = 0
write(1, "\320\240\320\276\320\264\320\270\321\202\320\265\320\273\321\214\321\201\320\272
\320\270\320\271 \320\277\321\200\320\276\321"... , 58Родительский процесс завершен.
) = 58
exit_group(0)                                   = ?
+++ exited with 0 +++

```

Вывод

В ходе выполнения лабораторной работы были освоены основы межпроцессного взаимодействия с использованием отображаемых файлов (Memory-Mapped Files). Реализован обмен данными между процессами через общий участок памяти, доступный всем участникам конвейера. Сигналы использовались для синхронизации этапов обработки. Работа продемонстрировала практическое применение механизмов совместной памяти и взаимодействия процессов в Linux.